



Rejestracja do grup

Silnik do realizacji przydziałów

Krzysztof Ciebiera, Marcin Mucha
Uniwersytet Warszawski

Spis treści

1	Wstęp	3
2	Wejście i wyjście	4
2.1	Uwagi ogólne	4
2.2	Struktura pliku wejściowego	4
2.2.1	Zajęcia	5
2.2.2	Grupy związane	6
2.2.3	Studenci	7
2.2.4	Przykładowy plik wejściowy	8
2.3	Błędy podczas wczytywania pliku wejściowego	9
2.4	Struktura pliku wyjściowego	11
2.4.1	Przykładowy plik wyjściowy	11
2.5	Dziennik wykonania programu	11
2.6	Raport	12
2.7	Wykorzystanie RDG w USOS	13
3	Opis działania programu	15
3.1	Klasa RDG	15
3.2	Klasa Optimizer	15
3.3	Symulowane wyżarzanie — SAOptimizer	16
3.4	Algorytm zachłanny — GreedyOptimizer	16
3.5	Parametry programu	17
4	Funkcja celu	18
4.1	Czynniki uwzględnione przy wyborze funkcji celu	18

4.2	Opis funkcji celu	19
4.2.1	Elementy funkcji celu	19
4.2.2	Sposób obliczania funkcji celu	19
5	Informacje dla studentów	21
5.1	Konflikty	21
5.2	Zestawy	22
5.3	Algorytm zachłanny	22

Rozdział 1

Wstęp

W tym dokumencie opisujemy silnik do realizacji przydziałów w procesie rejestracji do grup. Dokument jest podzielony następująco:

Rozdział 2 opisuje format wejścia i wyjścia programu, sposób jego uruchamiania oraz opis błędów sygnalizowanych przez program.

Rozdział 3 opisuje budowę programu.

Rozdział 4 opisuje funkcję celu stosowaną do oceny jakości znajdowanych przez program rozwiązań.

Rozdział 5 jest przeznaczony dla studentów, którzy mają pośrednio korzystać z programu.

Rozdział 2

Wejście i wyjście

2.1 Uwagi ogólne

- Program stosuje kodowanie UTF-8.
- Domyślnie program odczytuje dane z pliku `dane.txt`.
- Domyślnie program wypisuje wyniki swojego działania do pliku `wyniki.txt`.
- Logi programu są wypisywane do pliku `log.txt`.
- Raport programu jest zapisywany w pliku `raport.csv`.
- Jeśli nie podajemy typu danych, to mamy na myśli dziewięciocyfrowe liczby całkowite.
- Tablice zapisujemy w stylu pythonowym czyli w nawiasach kwadratowych po-oddzielane przecinkami. Przykład: `[[a,b,c],[d,e,f],[g,h]]`.
- Białe znaki w obrębie wiersza są dopuszczalne.
- Puste wiersze są dopuszczalne.
- Od znaku `#` zaczyna się komentarz, który trwa do końca wiersza. Komentarz jest pomijany przez program.

2.2 Struktura pliku wejściowego

Plik składa się logicznie z dwóch części. Pierwsza część pliku opisuje zajęcia i grupy, a druga opisuje preferencje osób. Części następują bezpośrednio po sobie.

2.2.1 Zajęcia

Pierwszy wiersz pliku zawiera liczbę zajęć k . Potem następuje opis k zajęć. Pojedynczy opis zajęć składa się z:

- liczby całkowitej, która jest jednoznacznym identyfikatorem zajęć (można tu użyć np. pola `zaj_cyk_id`, czyli klucza zajęć w tablicy `dz_zajecia_cykli`, które jednoznacznie identyfikuje zajęcia),
- zawartego w cudzysłowach opisu czytelnego dla człowieka¹,
- tablicy opisującej poszczególne grupy.

Każda grupa jest opisywana przez tablicę 4-elementową zawierającą:

- numer grupy (unikatowy w obrębie zajęć),
- limit miejsc w grupie,
- liczbę zajętych miejsc w grupie,
- tablicę terminów, w których odbywają się zajęcia tej grupy, każdy z elementów tej tablicy jest tablicą zawierającą 3 liczby o następujących intuicyjnych znaczeniach:
 - dzień tygodnia (poniedziałek = 1, wtorek = 2, ..., niedziela = 7),
 - pora rozpoczęcia w formacie hhmm, czyli 9:00 to 900, 20:09 to 2009,
 - pora zakończenia w formacie hhmm.

Kolizje zajęć

Grupy kolidują ze sobą jeśli są tego samego dnia tygodnia, a otwarte przedziały (pora rozpoczęcia, pora zakończenia) mają niepuste przecięcie. Jeśli grupy mają po kilka terminów, to kolidują ze sobą jeśli co najmniej jedna para terminów koliduje.

Liczba wolnych miejsc

W pliku wejściowym podawane są limit miejsc i liczba miejsc zajętych w grupie. Na podstawie tych dwóch informacji program oblicza liczbę wolnych miejsc w grupie. Program nie sprawdza czy liczba wolnych miejsc jest wystarczająca do przydzielenia zajęć wszystkim studentom, natomiast stara się przydzielić miejsca w taki sposób, aby nie przekroczyć narzuconych limitów. W przypadku gdy tego nie potrafi zrobić, stara się przekraczać limity w każdej grupie po równo (czyli w każdej z grup limit będzie

¹ Program nie oczekuje żadnej konkretnej formy tego opisu jedynie wymaga, aby opis ten był pojedynczym napisem (może to być np. kod cyklu dydaktycznego, kod przedmiotu i typ zajęć). W raportach opis ten będzie się pojawiał jako nazwa zajęć.

przekroczony o tyle samo osób). Program nie przekracza limitów proporcjonalnie do rozmiarów grup.

W szczególności, jeśli w pliku pojawiają się grupy z liczbą wolnych miejsc równą zero, to może się zdarzyć, że program i tak zapisze do tych grup studentów. Dlatego w momencie generowania pliku wejściowego nie należy w ogóle eksportować grup, których składy są ustalone z góry i zamknięte.

Informacje o przekroczonych limitach pojawiają się w raportach generowanych przez program.

Jeśli chcemy, aby studenci nie mogli zapisywać się do jakiejś grupy, ale z jakichś powodów chcemy ją umieścić w pliku wejściowym, to jako jej limit podajemy -1 . Żaden student nie powinien mieć tej grupy wpisanej w preferencjach.

Jeśli na przedmiocie jest za dużo miejsc, to program stara się równoważyć obciążenie grup proporcjonalnie do limitów.

2.2.2 Grupy związane

Po opisach poszczególnych zajęć następują opisy grup związanych. Pierwszy wiersz tej części pliku zawiera liczbę m będącą liczbą grup związanych. Każdy z następnych m wierszy opisuje pojedynczą grupę związaną. Opis grupy związanej, składa się z:

- numeru zajęć,
- numeru grupy,
- numeru zajęć związanych,
- tablicy zawierającej numery grup związanych.

Jeżeli grupa a jest związana z grupą b , to uważamy również, że grupa b jest związana z grupą a .

Jeśli choć jedna grupa należąca do zajęć A jest związana z grupą należącą do zajęć B , to wszystkie grupy należące do zajęć A mogą być związane tylko z grupami należącymi do zajęć B . Czyli np. nie dopuszczamy do związania ćwiczeń z SO z laboratoriami z SO i PW. Dopuszczamy natomiast możliwości związań jeden-do-wielu oraz wiele-do-wielu. Dopuszczamy również możliwość związania tylko części grup z danego przedmiotu.

Interpretacja związań

Jeśli grupa a z zajęć A jest związana z grupami $b_1, b_2, \dots, b_k$ z zajęć B , to jeśli program zapisze studenta do grupy a , to nie zapisze studenta do żadnej grupy z B oprócz $b_1, b_2, \dots, b_k$. Intuicyjnie mówiąc, program nie pozwoli studentowi złamać związań. Może się zdarzyć jednak, że program zapisze studenta do grupy a i nie zapisze go do żadnej z grup $b_1, b_2, \dots, b_k$. Może tak się stać w sytuacji, kiedy student zapisał się

na zajęcia A , a nie zapisał się na zajęcia B . Podobnie może się zdarzyć, że program zapisze studenta do jednej z grup $b_1, b_2, \dots, b_k$ nie zapisując go do grupy a .

2.2.3 Studenci

Po opisie zajęć następują opisy preferencji studentów. Pierwszy wiersz tej części pliku zawiera liczbę n będącą liczbą studentów. Następne n wierszy to opisy poszczególnych studentów. Opis studenta składa się z:

- numeru studenta,
- zawartego w cudzysłowach opisu studenta czytelnego dla człowieka,
- wagi studenta (co najwyżej dziewięciocyfrowa liczba całkowita),
- tablicy zajęć, na które chce się zapisać student,
- tablicy preferencji (format preferencji jest opisany poniżej),
- tablicy zajęć, z których student dopuszcza kolizje,
- tablicy terminów, które spowodują kolizje (w takim formacie jak terminy grup).

Każdy student może być opisany, podobnie jak zajęcia, czytelnym dla człowieka napisem (np. PESEL, imię, nazwisko).

Opis preferencji

Opis preferencji jest tablicą pojedynczych zestawów. Zestawy pojawiające się w tej tablicy na wcześniejszych pozycjach są bardziej preferowane przez studenta.

Pojedynczy zestaw to lista grup z poszczególnych zajęć, na które student chce się zapisać. Przykładowo, jeśli student zapisał się na zajęcia $[1, 2, 3]$, to jego preferencje $[[11, 12], [22], [31, 32]]$ oznaczają, że z zajęć pierwszych chce dostać się do grup 11 lub 12, z zajęć 2 do grupy 22, a z zajęć 3 do grup 31, 32.

Dopuszczamy również puste listy grup. Pusta lista oznacza, że studentowi odpowiada dowolna grupa odpowiednich zajęć. W sytuacji z poprzedniego przykładu opis $[[11, 12], [], []]$ zostanie zinterpretowany jako: grupa 11 lub 12 na pierwszych zajęciach oraz dowolne grupy na pozostałych zajęciach.

Tablica dopuszczalnych kolizji

Każdy student może mieć określoną tablicę zajęć, z których dopuszcza kolizję. Podaje się ją w postaci listy numerów zajęć. Lista ta może być pusta.

Tablica kolidujących terminów

Każdy student może mieć określoną listę terminów, które powodują kolizję. Format jak również interpretacja tych terminów jest identyczna jak w punkcie 2.2.1. W ten sposób można przekazać programowi informacje o zajęciach studenta, które nie są zawarte w rejestracji (np. WF, lektoraty, etc.).

Interpretacja konfliktów

Program stara się zminimalizować całkowitą liczbę konfliktów (sumę liczby konfliktów poszczególnych studentów). Za konflikt uważamy każdą parę grup, na które zapisaliśmy studenta, których terminy są ze sobą w konflikcie.

Jeśli dwie grupy są w tym samym terminie, to konflikt jest jeden, jeśli trzy grupy są w tym samym terminie to konfliktów jest trzy, jeśli cztery grupy są w tym samym terminie, to konfliktów jest 10.

Interpretacja wag

Wagi studentów są mnożnikiem kary, którą program daje rozwiązaniu dla konkretnego studenta. Znaczenie tej kary jest zależne od tego jakie są parametry programu, ale w domyślnych ustawieniach można je interpretować następująco:

Jeśli jeden student ma wagę x razy większą niż drugi, to dla programu rozwiązanie, które drugiemu studentowi daje x konfliktów jest równie dobre, jak rozwiązanie dające pierwszemu studentowi jeden konflikt. Waga, w analogiczny sposób, dotyczy preferencji studentów.

2.2.4 Przykładowy plik wejściowy

```
2 # dwa zajecia
# zajecia o zaj_cyk_id=105374, trzy grupy
105374 "1000-215aSOP,2008Z,CW"
# grupa 1, limit 15, poniedziałek 10:00-12:00 i sroda 10:00-12:00
# grupa 2, limit 10, poniedziałek 10:00-12:00
# grupa 3, limit 10, wtorek 12:00-13:00
[[1, 15, 0, [[1, 1000, 1200],[3,1000,1200]]],
 [2, 10, 0, [[1, 1000, 1200]]], [3, 10, 0, [[2, 1200, 1300]]]]
# zajecia o zaj_cyk_id=105386, trzy grupy
105386 "1000-225aSOP, 2008Z, LAB"
# grupa 1, limit 15, poniedziałek 10:00-12:00 i sroda 10:00-12:00
# grupa 2, limit 10, poniedziałek 10:00-12:00
# grupa 3, limit 10, wtorek 12:00-13:00
[[1, 15, 0, [[1, 1000, 1200],[3,1000,1200]]],
 [2, 10, 0, [[1, 1000, 1200]]], [3, 10, 0, [[2, 1200, 1300]]]]
3 # trzy grupy zwiazane
```

```
105374 1 105386 [1]
# zajecia o zaj_cyk_id=105374 grupa 1 pozwala chodzic na jedna grupe:
# zajecia 105386 grupa 1
105374 2 105386 [2]
# zajecia o zaj_cyk_id=105374 grupa 2 pozwala chodzic na jedna grupe:
# zajecia 105386 grupa 2
105374 3 105386 [3]
# zajecia o zaj_cyk_id=105374 grupa 3 pozwala chodzic na jedna grupe:
# zajecia 105386 grupa 3

# Część II, OSOBY
3 # trzy osoby
34267 "Jan Kowalski, 1234567890"
# osoba o os_id=34267 chodzi na zajecia 105374, waga 5,
  preferencje grupa 1 i grupa 2
5 [105374] [[[1]], [[2]]] [] []
467143 "Andrzej Nowak, 2345678901"
# osoba o os_id=467143 chodzi na zajecia 105386, waga 5,
  nie ma preferencji i dopuszcza kolizje na zajeciach 105386
5 [105386] [] [105386] []
88456 "Anna Kowalska, 456789012345"
# osoba o os_id=88456 i wadze 2 chodzi na zajecia 105374 i 105386,
# najbardziej chcialaby trafic do grup 1 1 i (2 1 lub 2 2),
# a jesli nie, to do (1 1 lub 1 2) i (2 2)
2 [105374, 105386] [[[1], [1, 2]], [[1, 2], [2]]] [] []
```

2.3 Błędy podczas wczytywania pliku wejściowego

Jeżeli program nie zostanie uruchomiony z parametrem będącym możliwym do otwarcia plikiem, to wypisze błąd: **BLAD: Nie moze otworzyc pliku <nazwa>**.

W trakcie wczytywania pliku wejściowego program może wygenerować jeden z następujących błędów:

- **BLAD L01: Ten identyfikator zajec zostal juz uzyty** — w pliku dwukrotnie wpisane są zajęcia o tym samym numerze.
- **BLAD L02: Do grupy zapisano juz wiecej osob niz wynosi jej limit** — liczba miejsc zajętych w grupie jest wyższa od limitu tej grupy.
- **BLAD L03: Dzień tygodnia powinien byc liczba z przedzialu 1..7** — dzień tygodnia nie jest z przedziału 1...7.
- **BLAD L04: Godzina powinna byc liczba z przedzialu 0000..2400** — godzina nie jest liczbą z przedziału 0...2400.

- **BLAD L05: W opisie zajec id_zajec powtarza sie identyfikator grupy** — ten sam identyfikator grupy jest uzyty wielokrotnie dla grup tych samych zajec.
- **BLAD L06: Bledny identyfikator zajec** — próba związania grupy z nieistniejącymi zajęciami.
- **BLAD L07: Zajecia id_zajec nie maja grupy id_grupy uzytej w definicji związania** — próba związania grupy o numerze, który nie znajduje się na liście numerów grup odpowiednich zajęć.
- **BLAD L08: Grupy tych samych zajec nie moga byc związane** — próba związania dwóch grup tych samych zajęć.
- **BLAD L09: Zajecia id_zajec1 zostaly juz wczesniej związane z zajeciami id_zajec2** — próba związania jednych zajęć z różnymi zajęciami.
- **BLAD L10: Grupa id_grupy1 zajec id_zajec1 i grupa id_grupy2 zajec id_zajec2 sa juz związane** — ponowna definicja związania tej samej pary grup.
- **BLAD L11: Identyfikator studenta id_studenta zostal juz uzyty** — dwóch studentów ma ten sam numer.
- **BLAD L12: Brak zajęć o numerze id_zajec** — próba zapisania studenta na nieistniejące zajęcia.
- **BLAD L13: Student id_studenta jest dwa razy zapisany na zajecia id_zajec** — na liście zajec studenta te same zajecia pojawiaja sie dwukrotnie.
- **BLAD L14: Zestaw preferencji (studenta id_studenta) powinien zawierac po jednej liscie grup dla kazdych zajec studenta** — liczbę elementów w zestawie preferencji nie zgadza się z liczbą zajęć, na które ma być zapisany student.
- **BLAD L16: Student id_studenta posiada w preferencjach zajec id_zajec nieistniejaca grupe id_grupy** — w zestawie preferencji studenta pojawia sie numer nieistniejącej grupy, tj. grupy nie wymienionej na liście grup odpowiednich zajęć.
- **BLAD L17: Student id_studenta deklaruje bezkolizyjnosc zajec id_zajec na ktore nie jest zapisany** — na liście zajęć bezkolizyjnych studenta pojawiają się zajęcia, na które nie jest on zapisany.
- **BLAD L18: Opis zablokowanego terminu nr nr_terminu u studenta id_studenta powinien skladac sie z dokladnie 3 elementow** — opis zablokowanego terminu powinien zawierac numer dnia oraz godziny rozpoczęcia i zakończenia, ten bład pojawia się jeśli opis ten zawiera więcej lub mniej niż 3 elementy.

2.4 Struktura pliku wyjściowego

Każdy z wierszy pliku opisuje jednego studenta. Opis pojedynczego studenta składa się z numeru tego studenta oraz tablicy par liczb (z , g), z których każda para oznacza, że na zajęcia z student powinien zostać zapisany do grupy g .

2.4.1 Przykładowy plik wyjściowy

Poniżej przedstawiony jest plik wyjściowy generowany przez program dla przykładu z [2.2.4](#).

```
34267 [[105374,1] ]
467143 [[105386,3] ]
88456 [[105374,1] , [105386,1] ]
```

Zwróćmy uwagę na to, że wszyscy studenci dostali się na wybrane przez siebie zestawy.

2.5 Dziennik wykonania programu

W trakcie działania program generuje dziennik, który pozwala administratorowi zorientować się jak dobre rozwiązanie program znalazł dotychczas. W dzienniku znajdują się informacje o najlepszym rozwiązaniu znalezionym przez program. Te informacje to:

- całkowita kara, im mniejsza tym rozwiązanie lepsze, najmniej może wynosić ona 0 (SCORE),
- kara za konflikty (CONFLICTS),
- kara za niespełnienie preferencji, całkowita liczba preferencji, liczba studentów którzy złożyli preferencje (PREFERENCES),
- kara za przekroczenie limitów grup (LIMITS),
- kara za niezrealizowanie związków (CONSTRAINTS).

Informacje o rozwiązaniu są wypisywane dla:

- rozwiązania początkowego (PRIMARY SCORE), które program wygenerował przed rozpoczęciem procesu optymalizacji (tylko dla algorytm symulowanego wyżarzania),
- rozwiązań pośrednich (NEW SCORE), które program wypisuje co 100000 rozpatrywanych rozwiązań,
- rozwiązania końcowego (SCORE).

W przypadku użycia algorytmu symulowanego wyżarzania dla każdego z rozwiązań pośrednich podawana jest również temperatura (T), która jest specjalną zmienną występującą we tym algorytmie.

Przykładowy dziennik może wyglądać następująco.

```
PRIMARY SCORE: 2.1
CONFLICTS: 0
PREFERENCES: 0.5/4 (2)
LIMITS: 0
CONSTRAINTS: 2
NEW SCORE after: 100000
CONFLICTS: 0
PREFERENCES: 0/4 (2)
LIMITS: 0
CONSTRAINTS: 0
T: 0.159513
NEW SCORE after: 200000
CONFLICTS: 0
PREFERENCES: 0/4 (2)
LIMITS: 0
CONSTRAINTS: 0
T: 0.00530022
.....
SCORE: 0
CONFLICTS: 0
PREFERENCES: 0/4 (2)
LIMITS: 0
CONSTRAINTS: 0
```

Widać, że program na początku znalazł rozwiązanie, które łamało dwa związania i nie spełniało wszystkich preferencji, a na końcu znalazł rozwiązanie idealne.

2.6 Raport

Raport jest w formacie CSV i składa się z trzech sekcji:

1. Raport wstępny — czas uruchomienia, parametry silnika, podstawowe informacje o danych wejściowych.
2. Raport szczegółowy — szczegółowe informacje o przydziałach studentów, konfliktach, zajętości grup.
3. Raport końcowy — ocena jakości rozwiązania

Przykładowy raport:

```
#,Raport wstepny

#,Data i czas uruchomienia: 09.12.21 09:59:21.
#,Parametry silnika:
#,Temperatura poczatkowa: 1
#,Waga preferencji: 0.2
#,Liczba zajec: 2
#,Liczba grup: 6
#,Liczba studentow: 3

#,Raport szczegolowy

#,Lista studentow

#,Id,Konfliktow,Wybrany zestaw,Liczba zestawow,Student
34267,0,1,2#,"Jan Kowalski, 1234567890"
467143,0,0,0#,"Andrzej Nowak, 2345678901"
88456,0,1,2#,"Anna Kowalska, 456789012345"

#,Liczba studentow z zestawami: 2
#,Liczba studentow z niezrealizowanymi zestawami: 0
0#,Liczba złamanych związań:
#,Lista grup

#,Przedmiot,Id,Grupa,Limit,Zajetych
#,"1000-215aSOP,2008Z,CW",105374,1,15,2
#,"1000-215aSOP,2008Z,CW",105374,2,10,0
#,"1000-215aSOP,2008Z,CW",105374,3,10,0
#,"1000-225aSOP, 2008Z, LAB",105386,1,15,2
#,"1000-225aSOP, 2008Z, LAB",105386,2,10,0
#,"1000-225aSOP, 2008Z, LAB",105386,3,10,0

#,Raport koncowy

#,Liczba konfliktow: 0
#,Liczba grup z przekroczonym limitem: 0
#,Liczba studentow bez konfliktow:3
#,Liczba studentow z konfliktami:0
```

2.7 Wykorzystanie RDG w USOS

Ogólny mechanizm przygotowywania pliku wejściowego może być następujący: Należy wziąć wszystkie zapisy (czyli pary student, zajęcia), na które student jest zapisany,

ale nie jest zarejestrowany na nie do żadnej grupy. Należy zdecydować czy powinny być one wzięte z jednej rejestracji, czy z wielu (jeśli np. się równolegle kilka odbywa). Potem należy je pogrupować po studentach i uzyskamy w ten sposób dla każdego studenta listę zajęć, na które on się ma zapisać. Potem dla studenta należy wziąć listę terminów, które są dla niego konfliktowe i w ten sposób uzyskamy opis studenta (oprócz preferencji).

Zajęcia można wypisać wszystkie. Programowi nie przeszkadza, że na któreś zajęcia nie chce nikt chodzić.

Niepełnosprawnym studentom, w domyślnych ustawieniach programu, należy dawać wagę dziesięć razy większą niż pozostałym. Wtedy dla programu ważniejsze będzie żeby zapisać studenta niepełnosprawnego na jakikolwiek wybrany przez niego zestaw niż żeby uniknąć konfliktu u studenta pełnosprawnego.

Rozdział 3

Opis działania programu

Działanie programu składa się z następujących kroków:

1. Wczytanie danych wejściowych. Dane wczytywane są do obiektu klasy RDG z pliku. W trakcie wczytywania danych odbywa się sprawdzenie ich poprawności składniowej.
2. Utworzenie obiektu pewnej podklasy klasy Optimizer, która będzie służyła do znalezienia najlepszego rozwiązania.
3. Szukanie rozwiązania funkcji solve() z odpowiedniej podklasy.
4. Wypisanie rozwiązania.
5. Wygenerowanie raportów szczegółowych.

W punkcie 3 w zależności od wybranego algorytmu używana jest klasa SAOptimizer (symulowane wyżarzanie) lub klasa GreedyOptimizer (algorytm zachłanny).

3.1 Klasa RDG

Obiekt klasy RDG reprezentuje plik wejściowy. Zawiera on opisy:

- zajęć (struct Course),
- grup, limitów i związań (struct Group),
- studentów i ich preferencji (struct Student).

3.2 Klasa Optimizer

Obiekt klasy Optimizer reprezentuje pojedynczy przebieg optymalizacji. Zawiera on struktury pozwalające na szybkie obliczanie funkcji celu i inne funkcje pomocnicze.

Z klasy tej dziedziczą klasy odpowiadające konkretnym metodom optymalizacji: SAOptimizer i GreedyOptimizer.

3.3 Symulowane wyżarzanie — SAOptimizer

Pierwszą z dostępnych metod optymalizacji jest symulowane wyżarzanie. Jeśli zostanie wybrana ta metoda, to program szuka rozwiązania minimalizującego funkcję celu, opisaną w rozdziale 4 przy użyciu metody symulowanego wyżarzania¹.

Parametry wyżarzania są następujące:

Parametr	Wartość
Temperatura początkowa	1
Temperatura końcowa	0.0001
Mnożnik	zależny od parametrów wejściowych
Liczba iteracji w stałej temperaturze	1

Tabela 3.1: Parametry wyżarzania

Pojedynczym ruchem w ramach algorytmu wyżarzania jest przepisanie studenta z jednej grupy do drugiej. Jest ono dokonywane za pomocą sekwencji:

- Student::tryMove()
- Student::commit()

3.4 Algorytm zachłanny — GreedyOptimizer

Drugą dostępną metodą jest algorytm zachłanny. Jeśli zostanie wybrana ta metoda, to studenci będą przydzielani do grup w kolejności malejących wag. Każdy kolejny student otrzyma przydział, który optymalizuje odpowiadającą mu część funkcji celu opisanej w rozdziale 4. Oznacza to z najczęściej, że program będzie starał się przypisać studenta w sposób pasujący do jak najlepszego zestawu spośród zestawów zdefiniowanych przez studenta, unikając przy okazji konfliktów między zajęciami. Jeśli jest to niemożliwe, to program używa wspomnianej funkcji celu przy podejmowaniu decyzji o tym, czy ważniejsze jest nieprzekraczanie limitów grup, czy może unikanie kolizji.

Warto w tym miejscu zaznaczyć, że dość często program ma do wyboru wiele równoważnych (z punktu widzenia studenta) przydziałów. Program stara się, w miarę możliwości czasowych, podejmować w takich sytuacjach decyzje w taki sposób, aby globalna funkcja celu miała jak najmniejszą wartość, t.j. m.in. aby liczba przekroczeń limitów i kolizji była jak najmniejsza.

¹http://en.wikipedia.org/wiki/Simulated_annealing

Aby uzyskać ten efekt stosuje się kilka różnych metod:

- Wybierając między grupami o tych samych terminach preferujemy grupy mniej zajęte.
- Wykonujemy wiele pełnych przydziałów do grup (w miarę możliwości czasowych, przy dużych testach i małych limitach czasowych nie jest to możliwe). Zbieramy statystyki dotyczące przekroczeń limitów grup, w kolejnych przydziałach staramy się w miarę możliwości unikać grup, które we wcześniejszych przydziałach były przepełnione.
- Wybierając pomiędzy grupami, którym poprzednie dwie heurystyki przypisują podobne wartości, korzystamy z randomizacji. W ten sposób program ma szansę znaleźć dobre rozwiązanie nawet jeśli heurystyki te zawiodą.

3.5 Parametry programu

Program jest uruchamiany za pomocą skryptu `rdg.sh`. W skrypcie tym zdefiniowane są następujące parametry.

Parametr	Wartość
ALGO	rodzaj algorytmu (sa lub greedy)
INPUT	nazwa pliku wejściowego
OUTPUT	nazwa pliku wyjściowego
LOG	nazwa pliku z dziennikiem
REPORT	nazwa pliku z raportem
TIME	czas działania programu w sekundach
PREF_GAP	kara za niezrealizowanie żadnej z preferencji (patrz p. 4.2.2)

Tabela 3.2: Parametry programu

Rozdział 4

Funkcja celu

4.1 Czynniki uwzględnione przy wyborze funkcji celu

Przy wyborze postaci funkcji celu wzięliśmy pod uwagę następujące czynniki:

Funkcjonalność: Funkcja celu powinna przede wszystkim oceniać jakość rozwiązania. Mamy pewne intuicje, które mówią o tym czy rozwiązanie jest lepsze czy też gorsze. Opisujemy je w rozdziale 4.2.

Przejrzystość dla studentów: Ze względu na to, że opis sposobu działania algorytmu optymalizującego będzie dostępny dla studentów, należy zadbać o to, aby sama funkcja oceniająca miała zrozumiałą postać. W szczególności powinno być jasne w jaki sposób poszczególne cechy rozwiązania wpływają na jej wartość.

Własności teorio-growe: W ramach rejestracji do grup studenci biorą udział w pewnego rodzaju grze. Charakter tej gry jest w pewnym stopniu zależny od postaci funkcji celu. Projektując funkcję celu należy zadbać o to, aby jej postać nie skłaniała studentów do działań niepożądanych, nie zmuszała do niepotrzebnej pracy, itp.

Elastyczność: Warto zaprojektować funkcję w sposób możliwie ogólny i elastyczny, co ułatwi ewentualne zmiany i adaptacje programu.

Koszt obliczeniowy: Należy unikać funkcji, których wyliczanie jest bardzo kosztowne obliczeniowo. Obliczanie wartości funkcji celu dla aktualnego rozwiązania jest często wąskim gardłem iteracyjnych algorytmów optymalizacyjnych. W szczególności, z reguły algorytmy takie w każdym kroku modyfikują rozwiązanie tylko nieznacznie i pożądane jest, aby w takiej sytuacji wartość funkcji celu dla nowego rozwiązania mogła być obliczona szybko.

Program stara się znaleźć rozwiązanie, które minimalizuje wartość funkcji celu.

4.2 Opis funkcji celu

4.2.1 Elementy funkcji celu

Funkcja celu uwzględnia (w kolejności od najważniejszych do najmniej ważnych):

Związania: Program przestrzega semantyki zdefiniowanej za pomocą grup związanych (2.2.2).

Limity grup: Program stara się, jeśli to możliwe, nie przekraczać limitów grup, wprowadzonych w danych wejściowych. Jeśli jest to konieczne limity są przekraczane równomiernie.

Kolizje: Program układa plany zajęć studentom w sposób minimalizujący liczbę kolizji.

Preferencje: Program stara się przydzielać studenta do grup w sposób przez niego preferowany.

4.2.2 Sposób obliczania funkcji celu

Funkcja celu, jest sumą kilku typów kar.

- Kara za złamanie jednego związania: 4.
- Kara za przekroczenie limitu grupy: $2 * nadmiar^2$.
- Kara za pojedynczy konflikt (zob. 2.2.3): $1 * waga$.
- Kara za niezrealizowanie żadnej z preferencji: $0.2 * waga$.
- Kara za zrealizowanie nie najlepszej preferencji: $0.2 * (1 - PREF_GAP) * \frac{\text{numer zestawu}-1}{\text{liczba zestawów}} * waga$.

W powyższych wzorach używamy następujących zmiennych:

- *nadmiar* – o ile miejsc jest przekroczony limit grupy,
- *waga* – waga studenta,
- *PREF_GAP* – parametr konfiguracyjny, intuicyjnie odpowiada różnicy pomiędzy niewybraniem przez program żadnego zestawu a wybraniem najgorszego z możliwych zestawów (powinien być z przedziału $[0, 1]$, gdzie 1 oznacza że różnicy nie ma),
- numer zestawu – numer zestawu, który został przydzielony studentowi (indeksowany od 1),
- liczba zestawów – liczba zestawów, które zdefiniował student.

Stałe występujące w powyższych wzorach zostały dobrane w sposób w pewnym stopniu arbitralny, ale też oparty na przeprowadzonych eksperymentach. Wydaje się, że ich zmienianie może mocno zdestabilizować sposób funkcjonowania programu. Wyjątkiem jest stała `PREF_GAP`, którą można samodzielnie ustalić w pliku konfiguracyjnym.

Rozdział 5

Informacje dla studentów

Silnik Rejestracji Do Grup posługuje się technikami Sztucznej Inteligencji w celu znalezienia przypisania studentów do grup (przypisanie to będziemy nazywać rozwiązaniem), które jest najlepsze dla jak największej liczby studentów.

5.1 Konflikty

Najważniejszym kryterium wyboru rozwiązania jest liczba konfliktów. Jeśli system znajdzie dwa rozwiązania i pierwsze z nich daje mniejszą liczbę konfliktów niż drugie, to system wybierze pierwsze.

Liczba konfliktów jest obliczana w następujący sposób.

1. System odczytuje terminy wszystkich zajęć, których student nie oznaczył jako takich, z których dopuszcza konflikt.
2. Dla każdej pary zajęć system sprawdza, czy czasy trwania zajęć nie pokrywają się. Jeśli czasy te pokrywają się, to student ma pojedynczy konflikt.
3. System sumuje liczbę pojedynczych konfliktów.

Przykłady:

- BD wtorek 10:15-11:45 i ASD wtorek 10:15-11:45 dają jeden konflikt.
- BD wtorek 10:15-11:45, ASD wtorek 10:15-11:45, RPiS wtorek 10:15-11:45 dają trzy konflikty.

Widzimy następujące powody, dla których proponowany przez nas sposób liczenia liczby konfliktów jest lepszy od uznania za konflikt liczby grup, które należy przesunąć, aby pozbyć się kolizji.

1. „Sprawiedliwość społeczna” — osoba, która dostanie 5 przedmiotów we wtorek o 10 zostanie w jakimś sensie bardzo mocno „skrzywdzona” przez system. Używana przez nas metoda liczenia (występuje ona też w optymalizowanej przez nas funkcji celu) sprawia, że system „rozsmarowuje” konflikty po studentach i nikomu nie dzieje się bardzo duża krzywda.
2. Nawet nie traktując tego w charakterze „sprawiedliwości społecznej”, można argumentować, że usunięcie konfliktu wielokrotnego jest trudniejsze. W jaki sposób taki student miałby zdefiniować swoje wymagania co do giełdy? Jeśli po prostu powie, że chce zamienić wtorek 10:00 na dowolny inny termin to mogą mu się porobić nowe konflikty. Szansa teoretycznie jest mała, ale w praktyce pewnie wcale nie tak bardzo, bo niektóre terminy są prawdopodobnie na konkretnym roku w naturalny sposób przepełnione z powodu konstrukcji grafiku. Z kolei podzielenie całego tygodnia na 4 rozłączne podzbiory i szukanie alternatyw w odpowiednich podzbiorach będzie dużo mniej skuteczne.
3. Jeśli konflikt dwuprzedmiotowy nie zostanie usunięty, to student ma przynajmniej szansę chodzić w połowie na każde z zajęć, co jeszcze ma trochę sensu, chodzenie na 20% zajęć sensu nie ma zupełnie.

5.2 Zestawy

W obrębie rozwiązań, które dają tę samą liczbę konfliktów, system poszukuje takich rozwiązań, w których:

- jak najwięcej studentów ma spełniony którykolwiek zestaw, oraz
- spełnione zestawy mają jak najmniejsze numery.

Od ustawień parametrów silnika zależy jak duże znaczenie przy wyborze rozwiązania ma każdy z wymienionych czynników. Można jednak założyć, że przydzielenie studentowi jakiegokolwiek zestawu jest ważniejsze niż zmniejszenie numeru przydzielonego zestawu.

Wniosek: nie należy tworzyć zestawów, na które nie chcemy się zapisać.

Nie ujawniamy celowo dokładnych informacji o tym jak system traktuje rozwiązania, w których tyle samo osób ma spełniony jakikolwiek swój zestaw. System będzie starał się doprowadzić do tego, żeby zapisać studentów na zestawy o niższych numerach.

5.3 Algorytm zachłanny

Domyślnie system nie stosuje bezwzględnej preferencji studentów o wyższych priorytetach/wagach (wynikających z średniej ocen itp). Opcja taka jest jednak dostępna w programie i na niektórych wydziałach może być użyta. W takiej sytuacji program

przydziela studentów do grup w kolejności ich priorytetów/wag. Każdy kolejny student otrzymuje najlepszy możliwy przydział (mówimy, że przydzielamy studentów do grup w sposób "zachłanny"), ale taki, który nie przekracza limitów grup i nie powoduje kolizji. W przypadku, gdy jest to niemożliwe, zachowanie programu zależy od jego ustawień.

W przypadkach, w których istnieje wiele równie dobrych przydziałów dla konkretnego studenta, program stara się, tak jak poprzednio, optymalizować globalną liczbę konfliktów i spełnionych zestawów.